Gradient descent

Model-based Machine Learning

To get a ML algorithm we usually follow 3 steps:

- 1) **Pick** a **model** (e.g.: hyperplane, decision tree, …)
  - We want to learn specific parameters
- 2) **Pick** a **criterion** to **optimize** → the **objective function** (e.g.: training error)
- 3) **Develop** a **learning algorithm**
  - Should try and **minimize the criteria**
  - **With** an **heuristic or exact** approach

The case of LINEAR MODELS

Pick a model

Linear models are defined by the following model:

$$0 = b + \sum_{j=1}^{m} w_j f_j$$

The **bias** b **and the weights** w_j are the **parameters we want** to **learn**

Pick a criterion to optimize

We **define a function that we want to optimize**:

$$\sum_{i=1}^{n} \mathbb{I}[y_i(w \cdot x_i + b) \leq 0]$$

Where:

- $\mathbb{I}[x] = \begin{cases} 1 & \text{if } x = \text{True} \\ 0 & \text{if } x = \text{False} \end{cases}$ is the indicator function
- $w \cdot x_i + b = 0$ defines an hyperplane
- The weight vector w is perpendicular to the hyperplane
- We **define the prediction** $w \cdot x_i + b$ for **any example** x_i **that doesn't stay on the hyperplane**
  - It's **proportional** to the **perpendicular distance from** the **hyperplane**
  - Its **sign** indicates the **side of the hyperplane** the point stands at
- y_i is the **label (ground truth)**

💡 **The function counts the number of wrong predictions → 0/1 loss function**
(if label and prediction are the same - same signs - their dot product is positive, else negative)

Develop a learning algorithm

We **aim to minimize** the **number of errors** in prediction:

$$\arg\min_{w,b} \sum_{i=1}^{n} \mathrm{I}[y_i(w \cdot x_i + b) \leq 0]$$

Loss functions

We search for **2 main characteristics in a loss function**

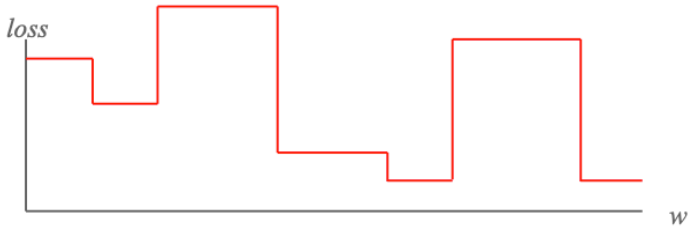
- It should be **continuous** and **differentiable** → to **get an indication of minimization direction**
- It should have **only one minima**

→ We search for **convex functions**

0/1 loss function

It's the **ideal loss function** → **minimizes** the **number** of **wrong predictions**

It's a **step function**



⚠️ It's **NOT a great loss function to minimize** (NP-hard)

- Is **not continuous** and is **not differentiable**
- Can have **multiple local minima**

💡 We should **search for a convex function that approximates it**

Surrogate loss function

It's a **proxy function that provides an upper bound on the actual loss function**

Some examples are:

- **Hinge loss function**: $\mathrm{L}(y, y') = \max(0, 1 - yy')$
- **Exponential loss function**: $\mathrm{L}(y, y') = \exp(-yy')$
- **Squared loss function**: $\mathrm{L}(y, y') = (y - y')^2$

Gradient descent

- One **approach to find the minimum** of a function → minimize the error function

Idea: partial **derivatives indicate** the **slope** in that dimension

- We **pick a starting point** w
- We **pick a dimension**
- We **move a small amount in that dimension → towards decreasing** loss
- **Repeat until** loss **doesn't decrease in any dimension**

$$w_j = w_j - \eta \frac{d}{dw_j} \text{loss}$$

Where:

- $\frac{d}{dw_j}\text{loss}$ is the **derivative**, it **indicates the function's slope** → max growth direction
  - **Positive derivative**
    - The **loss increases**
    - w_j should **follow** the **opposite direction** → we **subtract**
  - **Negative derivative**
    - The **loss decreases**
    - w_j should **follow** the **same direction** → we **add**
- η is the **learning rate**: defines **how much we want to move** in the error direction

Exponential loss function

Exponential loss function derivative

$$\begin{aligned}
\frac{d}{dw_j}\text{loss} &= \frac{d}{dw_j}\sum_{i=1}^{n} \exp(-y_i(w \cdot x_i + b)) \\
&= \sum_{i=1}^{n} \exp(-y_i(w \cdot x_i + b)) \cdot \frac{d}{dw_j}(-y_i(w \cdot x_i + b)) \\
&= \sum_{i=1}^{n} \frac{d}{dw_j} \exp(-y_i(w \cdot x_i + b)) \\
&= \sum_{i=1}^{n} -y_i x_{ij} \exp(-y_i(w \cdot x_i + b))
\end{aligned}$$

Exponential loss function update rule

$$w_j = w_j - \eta \frac{d}{dw_j} \text{loss}$$

$$w_j = w_j + \eta \sum_{i=1}^{n} y_i x_{ij} \exp(-y_i(w \cdot x_i + b))$$

Loss function generic update rule for each example

$$w_j = w_j + x_{ij} y_i \eta \exp(-y_i(w \cdot x_i + b))$$

Loss function generic update rule

$$w_j = w_j + x_{ij} y_i c$$

In the exponential loss function case:

$$c = \eta \exp(-y_i(w \cdot x_i + b))$$

- When label and prediction have the same sign → larger the prediction, smaller the update
- When label and prediction have different sign → larger the prediction, bigger the update

Perceptron algorithm with gradient descent

Algorithm 1 Perceptron with Gradient Descent

repeat until convergence (or for a fixed number of iterations)
  for all training examples $(f_1, f_2, \ldots, f_n, y)$ **do**
    $prediction \leftarrow b + \sum_{i=1}^{n} w_i f_i$
    $c \leftarrow \eta \exp(-y \cdot prediction)$
    for all w_i **do**
      $w_i \leftarrow w_i + c\, y\, f_i$ ▷ $w_i = w_i + \eta \exp(-y \cdot prediction)\ y\, f_i$
    end for
    $b \leftarrow b + c\, y$
  end for
until convergence

Gradient

Is the **vector of partial derivatives**

$$\nabla_w L = \left[\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2} \dots \frac{\partial L}{\partial w_N}\right]$$

Each partial derivative **measures how fast the loss changes** in any direction

Vanishing gradient

When the **gradient is** 0**:**

- The **loss** is **not changing** in any direction
- There could be a **local minimum** (**non-convex error function**) or a **saddle point**
  - Exactly on the saddle point → we are stuck
  - Slightly to the side of the saddle point → we can get unstuck

⚠️ The problem occurs **when the gradient** (= the derivative) is an extremely **small value**

- We say that **the function tends to saturate**
- The **gradient based learning** becomes **slower and more difficult** to be applied

The learning rate is the most important hyper-parameter in gradient descent